

Федеральное государственное автономное образовательное учреждение высшего
образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 «Программная инженерия» –

Системное и прикладное программное обеспечение

Отчёт

По лабораторной работе №2

по предмету

Распределенные системы хранения данных

Вариант: 987425

Выполнили:

студенты 3 курса

Батманов Даниил Евгеньевич

Демидов Иван Алексеевич

Группа: Р3307

Принял:

Николаев Владимир Вячеславович

Отчёт принят «__»____2025 г.

Оценка: _____

г. Санкт-Петербург, 2025

Задание:

Цель работы - на выделенном узле создать и сконфигурировать новый кластер БД Postgres, саму БД, табличные пространства и новую роль, а также произвести наполнение базы в соответствии с заданием. Отчёт по работе должен содержать все команды по настройке, скрипты, а также измененные строки конфигурационных файлов.

Способ подключения к узлу из сети Интернет через helios:

```
ssh -J sXXXXXXX@helios.cs.ifmo.ru:2222 postgresY@pgZZZ
```

Способ подключения к узлу из сети факультета:

```
ssh postgresY@pgZZZ
```

Номер выделенного узла pgZZZ, а также логин и пароль для подключения Вам выдаст преподаватель.

Этап 1. Инициализация кластера БД

- Директория кластера: \$HOME/сах22
- Кодировка: ANSI1251
- Локаль: русская
- Параметры инициализации задать через переменные окружения

Этап 2. Конфигурация и запуск сервера БД

- Способы подключения: 1) Unix-domain сокет в режиме peer; 2) сокет TCP/IP, принимать подключения к любому IP-адресу узла
- Номер порта: 9425
- Способ аутентификации TCP/IP клиентов: по паролю в открытом виде
- Остальные способы подключений запретить.
- Настроить следующие параметры сервера БД:
 - max_connections
 - shared_buffers
 - temp_buffers
 - work_mem
 - checkpoint_timeout
 - effective_cache_size
 - fsync
 - commit_delay

Параметры должны быть подобраны в соответствии со сценарием OLAP:
9 одновременных пользователей, пакетная запись/чтение данных по 128МБ.

- Директория WAL файлов: \$HOME/ems92
- Формат лог-файлов: .csv
- Уровень сообщений лога: ERROR
- Дополнительно логировать: контрольные точки и попытки подключения

Этап 3. Дополнительные табличные пространства и наполнение базы

- Создать новые табличные пространства для различных таблиц: \$HOME/cei80, \$HOME/jax56, \$HOME/jer28
- На основе template0 создать новую базу: goodgoldlake
- Создать новую роль, предоставить необходимые права, разрешить подключение к базе.
- От имени новой роли (не администратора) произвести наполнение ВСЕХ созданных баз тестовыми наборами данных. ВСЕ табличные пространства должны использоваться по назначению.
- Вывести список всех табличных пространств кластера и содержащиеся в них объекты.

Выполнение:

Этап 1: Инициализация кластера БД

Подключение: `ssh -J s367868@helios.cs.ifmo.ru:2222 postgres1@pg105` (+ пароль от helios) (+ пароль от пользователя postgres1)

Установка переменных окружения и инициализация кластера:

`PGDATA=$HOME/cax22`

`PGLOCALE=ru_RU.UTF-8`

`PGENCODING=UTF8`

`PGUSERNAME=postgres1`

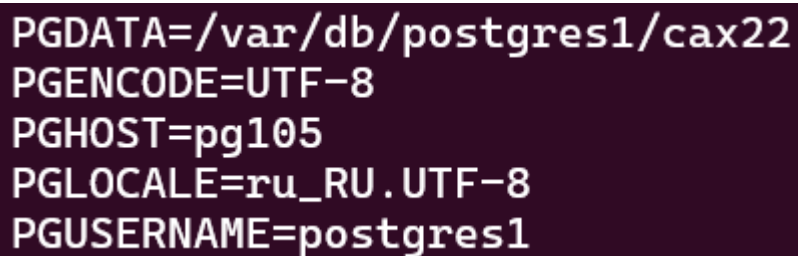
`PGHOST=pg105`

`PGWAL=$HOME/ems92`

`export PGDATA PGLOCALE PGENCODING PGUSERNAME PGHOST PGWAL`

* ANSI - это институт стандартов. По сути, не существует такой кодировки. Часто под ANSI понимают однобайтовую кодировку, выбранную в данный момент в системе пользователя. Но надеяться на то, что на машине пользователя будут точно те же региональные настройки, не стоит.

`env | sort`



```
PGDATA=/var/db/postgres1/cax22
PGENCODING=UTF-8
PGHOST=pg105
PGLOCALE=ru_RU.UTF-8
PGUSERNAME=postgres1
```

`initdb -D $PGDATA --encoding=$PGENCODING --username=$PGUSERNAME
--locale=$PGLOCALE --waldir=$PGWAL`

```
[postgres1@pg105 ~]$ initdb -D $PGDATA --encoding=$PGENCODING --username=$PGUSERNAME --locale=$PGLOCALE
Файлы, относящиеся к этой СУБД, будут принадлежать пользователю "postgres1".
От его имени также будет запускаться процесс сервера.

Кластер баз данных будет инициализирован с локалью "ru_RU.UTF-8".
Выбрана конфигурация текстового поиска по умолчанию "russian".

Контроль целостности страниц данных отключён.

исправление прав для существующего каталога /var/db/postgres1/cax22... ок
создание подкаталогов... ок
выбирается реализация динамической разделяемой памяти... posix
выбирается значение max_connections по умолчанию... 100
выбирается значение shared_buffers по умолчанию... 128MB
выбирается часовой пояс по умолчанию... Europe/Moscow
создание конфигурационных файлов... ок
выполняется подготовительный скрипт... ок
выполняется заключительная инициализация... ок
сохранение данных на диске... ок

initdb: предупреждение: включение метода аутентификации "trust" для локальных подключений
initdb: подсказка: Другой метод можно выбрать, отредактировав pg_hba.conf или ещё раз запустив initdb с ключом -A, --auth-local или --auth-host.

Готово. Теперь вы можете запустить сервер баз данных:

pg_ctl -D /var/db/postgres1/cax22 -l файл_журнала start
```

`pg_ctl -D /var/db/postgres1/cax22 -l logfile start`

```
[postgres1@pg105 ~]$ pg_ctl -D /var/db/postgres1/cax22 -l logfile start
ожидание запуска сервера.... готово
сервер запущен
```

Этап 2. Конфигурация и запуск сервера БД

Изменение файла ``postgresql.conf`` (путь: ``~/cax22/postgresql.conf``):

`listen_addresses = '*'`

`port = 9425`

`unix_socket_directories = '/var/run/postgresql'`

`max_connections = 20`

- В OLAP-системах обычно меньше активных соединений, но каждое соединение может выполнять тяжелые запросы.
- 9 пользователей + резерв для системных процессов = **20 соединений** (чтобы избежать перегрузки).
- Если поставить слишком много (>100), это увеличит накладные расходы на память.

`shared_buffers = 128MB`

- `shared_buffers` — это кэш PostgreSQL в оперативной памяти.
- Для OLAP важнее эффективное чтение больших объемов данных, но не обязательно держать всё в памяти.
- **128 МБ** — разумный минимум для небольших аналитических запросов.
- Можно увеличить до **25% от RAM**, если сервер выделен только под PostgreSQL.

`temp_buffers = 32MB`

- `temp_buffers` - память для временных таблиц и сортировок.
- В OLAP часто используются сложные запросы с `join`, `order by`, `group by`, которые создают временные структуры.
- **32 МБ** — достаточно для обработки данных в рамках 128 МБ пакетов.

`work_mem = 64MB`

- `work_mem` - память для операций сортировки и хеширования **на одно соединение**.
- В OLAP запросы часто включают сортировки больших таблиц.
- **64 МБ** позволяет обрабатывать **128 МБ данных** без избыточного использования диска.

- Если запросы очень тяжелые, можно увеличить до **128–256 МБ**, но это может привести к swap'у при многих соединениях.

`checkpoint_timeout = 900s`

- `checkpoint_timeout` — как часто PostgreSQL записывает изменения из WAL в основную БД.
- В OLAP редкие, но большие пакетные вставки → лучше реже, но крупные checkpoint'ы.
- **15 минут** — баланс между нагрузкой на диск и риском потери данных при сбое.

`effective_cache_size = 1GB`

- `effective_cache_size` — оценка, сколько памяти доступно для кэширования (включая `shared_buffers` и файловый кэш ОС).
- Для OLAP важно, чтобы большие запросы не упирались в диск.
- **1 ГБ** — разумное значение для сервера с **4+ ГБ RAM**.

`fsync = on`

- `fsync` гарантирует, что данные записаны на диск, а не только в кэш.
- В OLAP важна **надежность**, поэтому `fsync = on`.
- Если нужна максимальная скорость (но риск потери данных), можно отключить (`fsync = off`), но **не рекомендуется**.

`commit_delay = 100`

- `commit_delay` — задержка перед фиксацией транзакции, чтобы группировать запросы.
- В OLAP много пакетных вставок → группировка снижает нагрузку на диск.
- **100 мкс** — компромисс между задержкой и производительностью.

`wal_level = replica`

`max_wal_size = 1GB`

`min_wal_size = 80MB`

`log_destination = 'csvlog'`

`logging_collector = on`

`log_directory = 'log'`

`log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'`

`log_checkpoints = on`

`log_connections = on`

`log_min_messages = error`

`listen_addresses = '*'`

`port = 9425`

`unix_socket_directories = '/var/db/postgres1/cax22'`

`fsync = on`

`commit_delay = 100`

`wal_level = replica`

`log_destination = 'csvlog'`

```
logging_collector = on
log_directory = 'log'
log_checkpoints = on
log_connections = on
log_min_messages = error
```

Настройка аутентификации в 'pg_hba.conf':

```
# TYPE DATABASE USER ADDRESS METHOD
local all all peer
host all postgres 192.168.11.105/32 password
host all all 127.0.0.1/32 password
host all all ::1/128 password
```

```
# TYPE DATABASE USER ADDRESS METHOD
# "local" is for Unix domain socket connections only
local all all trust
# IPv4 local connections:
host all all 127.0.0.1/32 trust
# IPv6 local connections:
host all all ::1/128 trust
# Allow replication connections from localhost, by a user with the
# replication privilege.
local replication all trust
host replication all 127.0.0.1/32 trust
host replication all ::1/128 trust
local all all peer
host all all 127.0.0.1/32 password
host all all ::1/128 password
```

Создание WAL директории и запуск сервера:

```
mkdir ~/ems92
pg_ctl -D /var/db/postgres1/cax22 -l logfile start
```

```
[postgres1@pg105 ~]$ pg_ctl -D /var/db/postgres1/cax22 -l logfile start
ожидание запуска сервера.... готово
сервер запущен
```

```
psql -h localhost -p 9425 postgres -c "SHOW shared_buffers;"
```

```
[postgres1@pg105 ~]$ psql -h localhost -p 9425 postgres -c "SHOW shared_buffers;"
shared_buffers
-----
128MB
(1 строка)
```

```
psql -h localhost -p 9425 postgres -c "SHOW effective_cache_size;"
```

```
[postgres1@pg105 ~]$ psql -h localhost -p 9425 postgres -c "SHOW effective_cache_size;"
effective_cache_size
-----
4GB
(1 строка)
```

Этап 3: Дополнительные табличные пространства и наполнение базы

Создание директорий и табличных пространств:

```
mkdir ~/cei80 ~/jax56 ~/jer28
```

```
psql -c "CREATE TABLESPACE cei80 LOCATION '/var/db/postgres1/cei80';"
```

```
psql -c "CREATE TABLESPACE jax56 LOCATION '/var/db/postgres1/jax56';"
```

```
psql -c "CREATE TABLESPACE jer28 LOCATION '/var/db/postgres1/jer28';"
```

```
[postgres1@pg105 ~]$ psql -p 9425 -d postgres
psql (16.4)
Введите "help", чтобы получить справку.

postgres=# CREATE TABLESPACE cei80 LOCATION '/var/db/postgres1/cei80';
CREATE TABLESPACE
postgres=# CREATE TABLESPACE jax56 LOCATION '/var/db/postgres1/jax56';
CREATE TABLESPACE
postgres=# CREATE TABLESPACE jer28 LOCATION '/var/db/postgres1/jer28';
CREATE TABLESPACE
postgres=#
```

Создание БД и роли:

```
CREATE DATABASE goodgoldlake TEMPLATE template0 ENCODING 'UTF8';
```

```
CREATE ROLE analyst WITH LOGIN PASSWORD 'securepass';
```

```
GRANT CONNECT, CREATE ON DATABASE goodgoldlake TO analyst;
```

```
postgres=# CREATE DATABASE goodgoldlake TEMPLATE template0 ENCODING 'UTF8';
CREATE DATABASE
postgres=# CREATE ROLE analyst WITH LOGIN PASSWORD 'securepass123';
CREATE ROLE
postgres=# GRANT CONNECT, CREATE ON DATABASE goodgoldlake TO analyst;
GRANT
```


#	TYPE	DATABASE	USER	ADDRESS	METHOD
# "local" is for Unix domain socket connections only					
local	all		all		trust
# IPv4 local connections:					
host	all		all	127.0.0.1/32	trust
# IPv6 local connections:					
host	all		all	::1/128	trust
# Allow replication connections from localhost, by a user with the					
# replication privilege.					
local	replication		all		trust
host	replication		all	127.0.0.1/32	trust
host	replication		all	::1/128	trust
local	all	all		peer	
host	all	all	127.0.0.1/32	password	
host	all	all	::1/128	password	
host	all	analyst	192.168.11.105/32	password	

Наполнение базы данных (пример):

```
psql -U analyst -d goodgoldlake -p 9425
```

```
[postgres1@pg105 ~]$ psql -U analyst -d goodgoldlake -p 9425
Пароль пользователя analyst:
psql (16.4)
Введите "help", чтобы получить справку.

goodgoldlake=> █
```

```
CREATE TABLE sales111 (id SERIAL, data TEXT) TABLESPACE cei80;
```

```
CREATE TABLE reports111 (id SERIAL, content TEXT) TABLESPACE jax56;
```

```
CREATE TABLE logs111 (id SERIAL, entry TEXT) TABLESPACE jer28;
```

```
-- Вставка тестовых данных
```

```
INSERT INTO sales111 (data) SELECT generate_series(1,1000)::TEXT;
```

```
INSERT INTO reports111 (content) SELECT repeat('Report ', 1000) FROM
generate_series(1,1000);
```

```
INSERT INTO logs111 (entry) SELECT md5(random())::TEXT FROM
generate_series(1,1000);
```

Проверка табличных пространств:

Tablespace	Objects	
cei80	sales222 (table, goodgoldlake.public)	+
	sales222 (table, template0.public)	+
	sales222 (table, template1.public)	+
	sales222 (table, postgres.public)	+
	sales111 (table, goodgoldlake.public)	+
	sales111 (table, template0.public)	+
	sales111 (table, template1.public)	+
	sales111 (table, postgres.public)	+
jax56	reports111 (table, goodgoldlake.public)	+
	reports111 (table, template0.public)	+
	reports111 (table, template1.public)	+
	reports111 (table, postgres.public)	
jer28	logs111 (table, goodgoldlake.public)	+
	logs111 (table, template0.public)	+
	logs111 (table, template1.public)	+
	logs111 (table, postgres.public)	
pg_default	No objects	
pg_global	pg_database_datname_index (index, goodgoldlake.pg_catalog)	+
	pg_database_datname_index (index, template0.pg_catalog)	+
	pg_database_datname_index (index, template1.pg_catalog)	+
	pg_database_datname_index (index, postgres.pg_catalog)	+
	pg_database_oid_index (index, goodgoldlake.pg_catalog)	+
	pg_database_oid_index (index, template0.pg_catalog)	+
	pg_database_oid_index (index, template1.pg_catalog)	+
	pg_database_oid_index (index, postgres.pg_catalog)	+

WITH objects AS (

SELECT

t.spcname AS tablespace,

d.datname AS database,

c.relname AS object_name,

CASE c.relkind

WHEN 'r' THEN 'table'

WHEN 'i' THEN 'index'

WHEN 'S' THEN 'sequence'

WHEN 't' THEN 'toast table'

WHEN 'v' THEN 'view'

WHEN 'm' THEN 'materialized view'

WHEN 'c' THEN 'composite type'

WHEN 'f' THEN 'foreign table'

WHEN 'p' THEN 'partitioned table'

ELSE c.relkind::text

END AS object_type,

```

    n.nspname AS schema
FROM pg_tablespace t
JOIN pg_class c ON c.reltablespace = t.oid
JOIN pg_namespace n ON n.oid = c.relnamespace
JOIN pg_database d ON pg_tablespace_location(t.oid) NOT LIKE '%/%'
    OR (c.relkind <> 'S' AND pg_relation_filenode(c.oid) = 0)
    OR pg_relation_filepath(c.oid) IS NOT NULL
WHERE c.relname NOT LIKE 'pg_toast%'
)
SELECT
    t.spcname AS "Tablespace",
    COALESCE(string_agg(
        o.object_name || ' (' || o.object_type || ', ' || o.database || '.' || o.schema || ')',
        E'\n'
    ), 'No objects') AS "Objects"
FROM pg_tablespace t
LEFT JOIN objects o ON t.spcname = o.tablespace
GROUP BY t.spcname
ORDER BY t.spcname;

```

Результат

- Созданы табличные пространства `cei80`, `jax56`, `jer28`.
- База `goodgoldlake` содержит таблицы в соответствующих табличных пространствах.

Итоговые команды и изменения:

- Все конфигурационные изменения отражены в `postgresql.conf` и `pg\_hba.conf`.
- Созданы необходимые директории и табличные пространства.
- Наполнение данных выполнено от имени роли `analyst`.

В ходе выполнения данной лабораторной работы я научился создавать процедуры в PostgreSQL.